TP — Raspberry Pi Pico 2 : station météo

MicroPython, bus I2C, capteur BME280, écran OLED et filtrage par moyenne

Marc Nicollerat Marion Badi

1 octobre 2026

Objectifs du TP

À l'issue de ce TP, l'étudiant · e est capable de :

1. Structurer un script MicroPython (imports, configuration, fonctions, boucle principale).
2. Mettre en œuvre un bus **I2C** partagé par plusieurs périphériques et identifier leurs adresses.
3. Utiliser un pilote (module Python) fourni pour lire un capteur de température et de pression (**BME280**).
4. Piloter un écran **OLED** 128 × 64 pour afficher des mesures formatées.
5. Assembler ces briques dans une application complète : `meteo_station.py`.
6. (Approfondissement) Caractériser le bruit d'une mesure et le réduire par un **filtrage par moyenne**.

Matériel et logiciels

— Un Raspberry Pi Pico 2 et son câble USB
— Un module **BME280** (ou BMP280) en I2C
— Un écran **OLED 0,96 SSD1306** 128 × 64 en I2C
— Une breadboard et des fils de liaison
— VS Code avec l'extension **MicroPico** (voir le TP précédent pour l'installation)

Fichiers fournis :

Copiez et dézippez ces fichier dans un dossier.

Fichier	Rôle
`bme280.py`	pilote du capteur BME280/BMP280 — **à copier sur le Pico**, ne pas modifier

Fichier	Rôle
`ssd1306.py`	pilote de l'écran OLED — **à copier sur le Pico**, ne pas modifier
`etape1_clignote.py`	template de la section
`etape2_bme280_fonction.py`	template de la section
`etape3_oled_fonction.py`	template de la section
`meteo_station.py`	template de l'application finale (sections et)
`etape5_bruit_pression.py`	template de l'approfondissement

Dans chaque template, les parties à compléter sont signalées par un commentaire `# TODO`.

Connexion et installation

Installation de micropython sur la carte (normalement déjà fait)

1. Maintenez le bouton **BOOTSEL** du Pico 2, branchez-le en USB, puis relâchez : il apparaît comme un disque `RP2350`.
2. Téléchargez le firmware MicroPython pour **Pico 2** depuis micropython.org/download et glissez le fichier `.uf2` sur le disque. Le Pico redémarre automatiquement.

Installation des outils dans VS Code

3. Dans VS Code, installez les extensions **Raspberry Pi Pico** et **MicroPico**.
4. Ouvrez le dossier de projet, lancez la commande (Shift-Ctrl-P) **"MicroPico : Initialize MicroPico project"**, puis connectez la carte.
5. En bas, l'onglet *Terminal* devrait afficher le REPL. Vérifiez que REPL répond (tapez `print("test")` directement dans le REPL).

> **!** Important
>
> Ne passez à la suite que lorsque le REPL répond correctement.

Faire clignoter la LED

Ouvrez `etape1_clignote.py` et complétez-le pour que la LED embarquée clignote avec une période de 1 s, et qu'un compteur incrémenté à chaque allumage soit affiché dans la console.

Fonctions à utiliser :

Appel	Rôle
`Pin("LED", Pin.OUT)`	crée un objet pour la LED embarquée, configurée en sortie
`led.on()` / `led.off()`	allume / éteint la LED
`time.sleep(s)`	attend `s` secondes (nombre à virgule accepté)
`print(...)`	envoie du texte sur la console (REPL de VS Code)

Question

Un programme de microcontrôleur se termine rarement : pourquoi la boucle `while True:` est-elle ici indispensable ? Que se passe-t-il si on l'interrompt avec **Ctrl+C** dans le REPL ?

Le bus I2C

Le capteur et l'écran communiquent tous deux par **I2C** : deux fils de signal (**SDA** pour les données, **SCL** pour l'horloge) sont connectés aux 2 périphériques. Chaque périphérique possède une **adresse** sur 7 bits qui permet au Pico de s'adresser à lui.

Les deux modules sont câblé sur les mêmes broches du PICO :

Signal module	Broche Pico 2	N° de broche physique
VCC / VIN	3V3(OUT)	36
GND	GND	38
SDA	GP0 (I2C0 SDA)	1
SCL	GP1 (I2C0 SCL)	2

Astuce

Vous pouvez vérifier que les deux périphériques répondent, en tapant dans le **REPL** :

```
from machine import I2C, Pin
i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
[hex(a) for a in i2c.scan()]
```

Vous devez obtenir deux adresses : `0x3c` (écran OLED) et `0x76` ou `0x77` (capteur).

Lire le capteur BME280

Le BME280 de Bosch mesure la température, la pression atmosphérique et l'humidité. Beaucoup de modules vendus comme « BME280 » sont en fait des **BMP280**, identiques mais **sans humidité** : ce TP n'utilise donc que la température et la pression.

Exploration dans le REPL

Avant d'écrire un script, testez les commandes **une par une dans le REPL**, en observant le résultat de chacune. Le bus `i2c` de la section doit exister.

1. **Importer** la classe du pilote :
   ```
   from machine import I2C, Pin
   from bme280 import BME280
   ```
2. **Initialiser** le capteur (créer l'objet) :
   ```
   i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
   capteur = BME280(i2c, adresse=0x76)
   ```
   Si une erreur apparaît, vérifiez l'adresse trouvée avec `i2c.scan()`.
3. **Récupérer** une mesure :
   ```
   mesure = capteur.lire()
   mesure
   ```
   `lire()` renvoie un **tuple** de trois valeurs : `(temperature, pression, humidite)`. Avec un BMP280, `humidite` vaut `None`. Affichez `capteur.a_humidite` pour savoir quel modèle vous avez.
4. **Décomposer** le tuple en variables :
   ```
   temperature, pression, humidite = capteur.lire()
   ```
5. **Formater** les valeurs pour l'affichage :

```
"T = {:.1f} C".format(temperature)
```

Écrivez de même une chaîne pour la pression, avec 2 décimales et l'unité hPa.

> **Questions**
>
> — Dans quelles unités le pilote renvoie-t-il la température et la pression ?
> — Que signifie `{:.1f}` ? Essayez `{:.3f}`, `{:6.1f}` et `{}`.
> — Essayez d'influencer le capteur : posez un doigt sur le capteur et relisez la température. Soufflez dessus….
> — La pression affichée est-elle celle annoncée par MétéoSuisse pour votre région ? Pourquoi peut-elle être nettement plus basse ? (indice : quelle est l'influence de la hauteur sur la pression ?)

Écriture d'un script avec des fonctions

Complétez `etape2_bme280_fonction.py`. Il contient deux fonctions à écrire :

Fonction	Rôle
`lire_meteo(capteur)`	lit le capteur et **retourne** `(temperature, pression)`
`formater_meteo(temperature, pression)`	**retourne** une chaîne du type `T = 23.4 C P = 954.12 hPa`

et une fonction `main()` qui crée le bus I2C et le capteur, puis affiche une mesure formatée toutes les 2 secondes.

Piloter l'écran OLED

L'écran SSD1306 comporte 128 × 64 pixels monochromes. Le pilote travaille avec une **image en mémoire** (un *framebuffer*) : on dessine d'abord dans cette image, puis on l'envoie d'un coup à l'écran avec `show()`. Tant que `show()` n'est pas appelé, rien ne change à l'écran.

Exploration dans le REPL

Testez les commandes suivantes une par une (le bus `i2c` doit exister) :

```
from ssd1306 import SSD1306_I2C
oled = SSD1306_I2C(128, 64, i2c, adresse=0x3C)
oled.text("Bonjour", 0, 0)
oled.show()
```

Puis explorez les méthodes de dessin :

Appel	Rôle
`oled.fill(c)`	remplit tout l'écran (`0` = noir, `1` = allumé)
`oled.text(chaine, x, y)`	écrit un texte, coin supérieur gauche en `(x, y)` ; police 8 × 8 pixels
`oled.hline(x, y, longueur, c)`	trace une ligne horizontale
`oled.rect(x, y, l, h, c)`	trace un rectangle vide (`fill_rect` pour un rectangle plein)
`oled.pixel(x, y, c)`	allume ou éteint un pixel
`oled.show()`	envoie l'image à l'écran

Questions

— Écrivez deux textes l'un par-dessus l'autre sans `fill(0)` entre les deux : que se passe-t-il ? Quel est le rôle de `fill(0)` ?
— Combien de caractères tiennent sur une ligne ? Combien de lignes sur l'écran ? Que se passe-t-il avec un texte trop long ?
— Où se trouve l'origine `(0, 0)` ? Dans quel sens l'axe `y` est-il orienté ?

Écriture d'un script avec des fonctions

Complétez `etape3_oled_fonction.py` :

Fonction	Rôle
`initialiser_oled(i2c)`	crée l'écran, l'efface et **retourne** l'objet
`afficher_message(oled, message)`	efface l'écran et affiche un message court
`afficher_mesures(oled, temperature, pression)`	affiche un titre, un trait, puis la température et la pression formatées

Le `main()` fourni teste votre affichage avec des **valeurs fictives** qui évoluent : vous pouvez ainsi mettre au point l'écran indépendamment du capteur.

Proposition de mise en page :

```
+----------------+
|Station meteo   |
|----------------|
|                |
|T:   23.4 C     |
|                |
|P:  954.12 hPa  |
|                |
+----------------+
```

> **Avertissement**
>
> La police intégrée ne contient pas les caractères accentués ni le symbole `°` : écrivez `C` plutôt que `°C`.

Application complète `meteo_station.py`

Assemblez les sections et dans le template `meteo_station.py` :

1. Recopiez vos fonctions `lire_meteo`, `initialiser_oled`, `afficher_message` et `afficher_mesures`.
2. Complétez `main()` :
   — création du bus I2C et affichage du résultat de `i2c.scan()` ;
   — initialisation de l'écran ;
   — création du capteur ;
   — message `Demarrage...` pendant une seconde ;
   — boucle : mesure, affichage sur l'écran **et** sur la console, attente de 2 s.

> **Pour que la station démarre seule**
>
> Un script nommé `main.py` dans la mémoire du Pico est **exécuté automatiquement à la mise sous tension**. Copiez `meteo_station.py` sur le Pico sous le nom `main.py`, puis alimentez le Pico par un chargeur USB (sans PC) : la station fonctionne de manière autonome.

Approfondissement : filtrage par moyenne

Observer le bruit de mesure

Affichez la pression toutes les 0,1 s avec 3 décimales, sans toucher au montage. La valeur fluctue sur les derniers chiffres : c'est le **bruit de mesure**. Une variation de 0,1 hPa correspond pourtant à un changement d'altitude d'environ **1 m** : ce bruit limite donc directement la résolution de la station (par exemple si on voulait l'utiliser comme altimètre).

Principe du moyennage

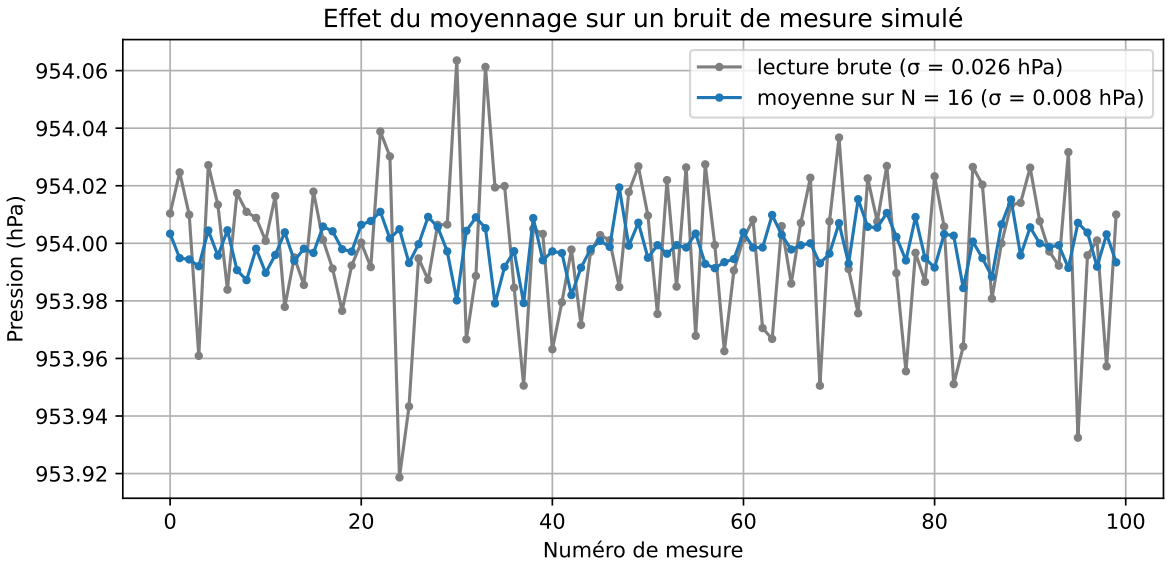
Si l'on effectue N mesures $x_1, \dots, x_N$ d'une grandeur constante, entachées d'un bruit aléatoire indépendant d'écart-type σ, leur moyenne

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i$$

a un écart-type réduit d'un facteur $\sqrt{N}$:

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{N}}$$

La simulation ci-dessous illustre cet effet sur un bruit de pression simulé :



> **Question**
>
> Pour diviser le bruit par 10, combien d'échantillons faut-il moyenner ? Si une lecture dure 10 ms, combien de temps dure alors une mesure ? Quel est le prix à payer du moyennage ?

Implémentation dans `meteo_station.py`

Écrivez la fonction :

```
def mesurer_pression_moyenne(capteur, nb_echantillons, delai_ms):
    ...
    return temperature, pression_moyenne
```

qui effectue `nb_echantillons` lectures du capteur, espacées de `delai_ms` millisecondes (`time.sleep_ms()`), et retourne la moyenne des pressions. Utilisez-la dans la boucle principale à la place de `lire_meteo()`, avec les constantes `NB_ECHANTILLONS_PRESSION` et `DELAI_ENTRE_ECHANTILLONS_MS` définies en tête du script.

> **Important**
>
> Écrivez votre propre fonction en utilisant `capteur.lire()` : n'utilisez pas la méthode `moyenne_pression()` du pilote.

Vérification expérimentale

Complétez `etape5_bruit_pression.py`, qui mesure deux séries de 50 valeurs de pression — l'une en lectures brutes, l'autre moyennées sur $N = 16$ — et calcule pour chacune la moyenne et l'écart-type :

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Comparez le rapport des deux écarts-types à la valeur théorique $\sqrt{16} = 4$.

Questions

- Refaites la mesure avec `DELAI_MS = 0`. Le rapport des écarts-types est-il toujours proche de 4 ? Affichez quelques lectures brutes successives : que remarquez-vous ?
- Le capteur produit une nouvelle conversion environ toutes les 7 ms (réglage du pilote). Expliquez pourquoi moyenner des lectures plus rapprochées que ce temps de conversion n'améliore pas la mesure.
- La moyenne élimine-t-elle une **erreur systématique** (un décalage constant de la pression mesurée) ? Justifiez.

Synthèse

Décrivez la chaîne complète de la station : grandeur physique → capteur BME280 → conversion et compensation dans le capteur → transfert I2C → pilote `bme280.py` → fonctions de lecture et de formatage → filtrage → affichage sur l'écran OLED. Pour chaque maillon, indiquez une source possible d'erreur ou de bruit.